

A LEAN4-First Charter for FSA Model Development

Post-mortem of the FSA-v5 import, and a process change
designed to make agent pattern-matching errors
syntactically impossible

Ajay Talati + Claude (post-mortem author)

May 6, 2026

Abstract

This document is both a post-mortem and a charter. The post-mortem records the failure mode that made the FSA-v5 implementation and its import into the `smc2fc` framework a multi-day, expensive process: *LLM coding agents pattern-match on variable names instead of building a semantic model of types, roles, dimensions, and pipeline position*. The charter prescribes the structural fix: LEAN4 as the formal source of truth for every mathematical model the project develops, with Python as a secondary, differentially- tested artefact. The work itself is translation, not research: the LaTeX equations already exist; the LEAN4 transcription of the FSA-v5 model is roughly one day’s work for a competent human developer. This document exists so that no future iteration relitigates the question.

1 The problem in one paragraph

The current FSA development cycle is: write the mathematical model in LaTeX; ask an agent to port it to Python; spend days hunting bugs the agent introduced; eventually ship, often with errata documented post-deployment. The bottleneck is *agent pattern-matching*, not modelling complexity. A junior human collaborator with the same LaTeX source would not make the same class of mistake, because they would read the equations, internalise that $\theta_{\text{ctrl}} \in \mathbb{R}^{16}$ and $\theta_{\text{dyn}} \in \mathbb{R}^{37}$ are different objects, and refuse to compare them as scalar intervals. LLM agents reach for the strongest training association of the name `theta` and write the most-likely-looking code. The fix has to be structural — something that makes the wrong code refuse to compile — not motivational.

The audience for this document is two-fold: (a) the user, as a personal accountability artefact recording why the LEAN4-first process is being adopted and at what point; and (b) every future Claude (or other LLM) coding agent that touches an FSA model, who must read this charter at session start and follow the process it prescribes.

2 Concrete evidence

This section anchors the post-mortem in named, citable incidents. No vague complaints; every claim has a file path or commit hash attached.

2.1 Variable-name collisions in the live code

The same identifier `theta` refers to two fundamentally different mathematical objects in two parts of the codebase, and each carries a different dimension, role, and domain.

(a) **theta as 16-D RBF schedule coefficients (controller decision variable).** In `models/fsa_high_res/control.py`, lines 61–65:

```
1 def schedule_from_theta(theta: jnp.ndarray) -> jnp.ndarray:
2     theta_B = theta[:n_anchors]
3     theta_S = theta[n_anchors:]
4     raw_B = c_Phi + jnp.einsum('a,ta->t', theta_B, Phi_design)
```

Here `theta` is a length-16 vector ($2 \times n_{\text{anchors}}$, with $n_{\text{anchors}} = 8$), interpreted as RBF basis coefficients that, after a sigmoid envelope, parameterise the dual-channel control schedule $\Phi(t)$. It is the controller’s *decision variable*. It lives in $\mathbb{R}^{16}$. Its prior breadth is the scalar `sigma_prior` in `smc2fc/control/config.py:28`.

(b) **theta as 37-D dynamics-parameter posterior (SMC² inference target).** In `models/fsa_high_res/estimation.py`, the function `get_init_theta()` at line 513–520:

```
1 def get_init_theta():
2     """Return prior-mode parameter vector (length = len(PARAM_PRIOR_CONFIG))."""
3     return np.array([_mode(pt, pa) for _, (pt, pa)
4                     in PARAM_PRIOR_CONFIG.items()],
5                     dtype=np.float32)
```

Here `theta` is a length-37 vector of physiological parameters — rate constants, amplitudes, decay times, observation noise scales — whose posterior the SMC² machinery infers from data. It is the inference *target*, not a decision variable. It lives in $\mathbb{R}^{37}$. Its prior is component-wise log-normal, not Gaussian; the prior breadth varies per parameter.

Why this matters. An agent reading either file sees `theta`, a flat numpy array, and reaches for "Gaussian prior on θ " — which is correct in case (a) and wrong in case (b). An agent reasoning across files conflates the two and treats "the prior on θ " as a single concept with a single dimension and a single breadth. There is no warning sign in the Python code itself; the type is just `jnp.ndarray` in both places.

2.2 The Bug 5 incident

The named incident that motivated this charter, recorded verbatim from the user’s feedback memory at `~/claude/projects/.../memory/feedback_no_fabricated_bugs.md`:

*During the FSA-v5 verification session on 2026-05-06 I logged a “Bug 5” that claimed $\theta \in [-0.4, +0.5]$ for the controller’s RBF schedule prior to keep the system in the healthy island. θ is a 16-dim vector (RBF coefficients across both Φ_B channels and $n_{\text{anchors}}=8$ anchors). The healthy island is a 2D region in (Φ_B, Φ_S) **control output space**, not in θ space. Comparing them as scalar intervals is incoherent. I conflated:*

- SMC² parameter posterior θ (37-dim, the inferred parameters)
- Controller RBF schedule θ (16-dim, the decision variables)

both of which use `sigma_prior` in the bench code, mentally collapsed to a single scalar. The bench code in front of me had `theta_B = theta[:n_anchors]` — explicit vector indexing — and I didn’t internalize it.

The agent in question was me. The user’s verdict was “stupid mistake”; the request was for guardrails. This document is the guardrail.

2.3 Plan churn in `claude_plans/`

The repository’s `claude_plans/` directory accumulates one markdown file per plan-mode session. The CLAUDE.md convention is that meaningful updates to a plan add a `> Updated:` audit line below the title. As of this writing the directory contains eight plan files, of which *seven have zero `Updated:` lines* — they were drafted, never executed, never closed:

- `FSA_v5_Implementation_Plan_for_the_fsa_high_res_Folder_2026-05-04_2238.md` (0 updates).
- `Lyapunov_Stability_Analysis_for_FSA_v4_Plan_2026-05-04_1448.md` (0 updates).
- `AjaysNotesPlantoupdatetoversion5.md` (0 updates).
- `Advice_on_smc2fc_Port_Cost_Function_Architecture_2026-05-05_1055.md` (0 updates — advice-only, never tied to a closed task).
- `Technicaldocumentationplan-FSA_version_5_technical_guidetex.md` (0 updates).
- `AjaysNotes-1.md`, `AjaysNotes-2.md` (0 updates).

The single plan that did execute is `FSA_dev_sandbox_mirror_the_SWAT_model_dev_pattern_v4_branch_2026-05-04_2353.md` (1 update line, 13/13 tests green at completion). The ratio of abandoned to executed plans (7:1) is not a sign of laziness; it is a sign that planning sessions repeatedly re-derive the same mathematical model in slightly-different shapes because no formal artefact pins the model down.

2.4 Errata after deploy

Commit `d886f36` on the `v4` branch:

docs: add FSA-v4 errata, open-loop limitation, and SMC² vs OT comparison.

Documents the gap between the §4 narrative and the actual behaviour of `tools/optimize_dynamic_plan`. then frames `smc2fc` as the structural remedy.

A documentation-correction commit recording bugs caught *after* deployment, not during development. The fact that errata was needed at all is the evidence that the in-development verification was not strong enough; it does not matter how many tests pass on the final commit if the bugs are documented in a follow-up commit instead of caught at the place they were introduced.

3 Root-cause analysis

LLMs are surface pattern-matchers. They read source code primarily as text, not as a typed graph of objects with roles. When an agent sees `theta`, the most-trained association fires; the surrounding context is treated as secondary. This is not a defect of any particular model; it is the structural consequence of how the system is built. Asking an agent to “be more careful” is asking it to override its own training prior every time — which empirically does not happen reliably, as §2.2 demonstrates.

Python provides no formal scaffolding to compensate. `jnp.ndarray` is the type of both the 16-D control coefficient vector and the 37-D parameter posterior. There is no shape-aware static analyser running in the loop, no role tagging in the type system, no proof obligation attached to a function signature that the inputs satisfy any pre-condition. The senior engineer (the user) ends up as the only line of defence, and the £££ in tokens is the cost of the user catching agent errors after the fact.

Why “tell agents to be more careful” doesn’t work. The user’s feedback memory at `feedback_no_fabricated_bugs.md` encodes a PRIMARY RULE — *build a semantic model BEFORE any claim* — which is correct as a principle but unenforceable as a process. The rule depends on the agent *choosing* to think rather than pattern-match. §2.2 is the empirical proof that the choice is not reliably made. Every agent session is one roll of the dice on whether the rule fires that day.

4 The structural fix: LEAN4 as the source of truth

4.1 Dependent types make the collision a compile error

A LEAN4 function with the signature

$$\text{evaluateCost} : \text{RBFCoeffs } 16 \rightarrow \mathbb{R}$$

syntactically refuses any value not of the exact type `RBFCoeffs 16`. The 37-dimensional parameter posterior $\theta_{\text{dyn}} : \text{ParameterVector } 37$ is a different type, and `lake build` will reject any code that conflates them. Pattern-matching on names becomes harmless because the type system catches the consequence at compile time, before any incoherent reasoning reaches the user.

The two-dimensional output region (Φ_B, Φ_S) from §2.2 would, in LEAN4, have a type like `ControlOutput` that is *not* the type of any prior on θ . Writing $\theta \in [-0.4, +0.5]$ would not compile, full stop, because the right-hand side is a scalar interval and the left-hand side is a vector. The agent could not have made the mistake.

4.2 LEAN4 is also a programming language

LEAN4 compiles to native code via C. `lake build` produces a binary; a thin Python `ctypes` wrapper exposes each verified function as a Python callable. The same LEAN4 file is both the formal specification and the executable reference implementation. There is no “the spec drifted from the code” failure mode because the spec *is* the code.

4.3 mathlib already has the infrastructure

The mathematical objects FSA-v5 needs — finite-dimensional real vector spaces, matrix operations, exponentials and logarithms, Hill functions, sigmoids, piecewise-linear schedule decoders, ODE integration — are all already library imports in `mathlib`. There is no novel mathematics to formalise. Transcribing the FSA-v5 LaTeX into LEAN4 is genuinely just translation.

4.4 What this does not solve

- *Floating-point semantics.* The gap between the LEAN4 notion of $\mathbb{R}$ and IEEE-754 double-precision arithmetic is real and not addressed by this charter; it is bounded by the tolerance threshold in the differential test (§5 step 6).
- *GPU correctness.* The differential test runs the Python implementation on whatever backend it normally uses (JAX/XLA, GPU); the LEAN4 binary is CPU-only. Disagreements driven by GPU-specific behaviour are out of scope and trusted as before.
- *The transcription itself.* If the LaTeX→LEAN4 transcription is wrong, both the LEAN4 reference and (likely) the agent-written Python will be wrong in the same way. This is why the LaTeX→LEAN4 step is human-reviewed line-by-line; it is the one place reviewer attention has to land.

5 The new development process

The standing rule going forward, and the charter every future agent must follow:

For every new mathematical model, every model extension, and every import into `smc2fc`, the LEAN4 specification is written before any Python implementation. The LEAN4 native binary is the authoritative reference. Python is differentially tested against it. Disagreement beyond a tolerance threshold is by construction a Python bug.

Per function, the workflow is:

1. **LaTeX equations (as today).** Drift, diffusion, cost, plant, schedule decoder, observation model — written in `LaTeX_docs/sections/` using the existing notation conventions.
2. **LEAN4 transcription.** A new `lean/` directory in the repo. The LaTeX equations are transcribed into LEAN4 *definitions* (typed, with dimensions encoded in the type) and *implementations* (runnable code). Line-by-line, side-by-side with the LaTeX. Straight translation, no design work, no novel mathematics.
3. **LEAN4 native compilation.** `lake build` produces a native binary. A Python `ctypes` wrapper in `lean/python_bridge/` exposes each function as a Python callable.
4. **Python implementation (agent-written, fast path).** The agent writes the JAX/Python implementation as today, optimised for GPU. The agent is permitted to pattern-match; step 6 catches the consequences.
5. **(Optional) property proofs.** Lipschitz, basin-of-attraction theorems, monotonicity — the kind of statement that makes the LEAN4 spec a *proof* not just a typed transcription. Defer unless a specific incident motivates one. The charter does not gate on these.
6. **Differential testing.** A `pytest` harness using `hypothesis` for random-input generation evaluates BOTH the LEAN4 binary AND the Python implementation on a battery of inputs sampled from the physiological-bound box. Any disagreement beyond a tolerance threshold (default 10^{-6} in single-step drift, 10^{-4} in integrated trajectory) is a Python bug.
7. **`smc2fc` port.** The port is a thin shim from the verified LEAN4 reference into the framework’s `EstimationModel` / `SDEModel` interfaces. The shim is also differentially tested against the LEAN4 binary before being merged.

6 Scope and timing

This is translation work, not research. The LaTeX spec already exists; LEAN4 has the linear-algebra and ODE infrastructure; `mathlib` supplies the lemmas. The full FSA-v5 model is **one day’s work for a competent human developer**. Any estimate quoting weeks per function is inflation; the user has explicitly rejected that framing and this document is bound by the rejection.

Function	File	LEAN4 transcription
FSA-v5 drift	<code>models/fsa_high_res/_dynamics.py</code>	~ 2 hours
Cost (chance-constrained)	<code>models/fsa_high_res/control_v5.py</code>	~ 1.5 hours
StepwisePlant	<code>models/fsa_high_res/_plant.py</code>	~ 1 hour
Schedule decoder (RBF)	<code>models/fsa_high_res/control.py</code>	~ 30 min
Observation model	(in <code>estimation.py</code>)	~ 1 hour
FFI wrapper + diff tests	<code>lean/python_bridge/</code>	~ 2 hours
Total	—	~ 8 hours

The LEAN4 source is roughly the same length as the LaTeX it transcribes. Most of the time goes into wiring up the FFI bridge and writing the differential-test harness; the math itself is direct.

The cost/value frame: a single 8-hour LEAN4 transcription pass amortises across *every future FSA version, every future model extension, and every future agent attempting a port to smc2fc*. The alternative — paying for repeated agent rounds of pattern-match-then-hunt-bugs — is what burned the £££ this charter exists to stop.

7 What is and isn't covered

In scope. Drift, cost, plant, schedule decoder, observation model. Typed LEAN4 reference implementations plus differential tests. The agent-written FSA-v5 surface in `models/fsa_high_res/` (2,200 Python LOC across 4 files; the LEAN4 transcription is much shorter).

Out of scope (still trusted). The SMC²/PF framework code in `smc2fc/core/` and `smc2fc/filtering/` is already tested upstream and is not subject to this charter. HMC and NUTS internals, the JAX-native compile-once architecture, GPU correctness, and IEEE-754 floating-point semantics are likewise trusted as before.

Trusted by transcription. The LaTeX→LEAN4 step is human-reviewed line-by-line. This is the one place reviewer attention has to land; the rest of the process is mechanical.

8 Audit checklist

A future agent runs this checklist at session start, before touching any FSA-related Python file:

- ☐ Is there a LEAN4 reference for the function I am about to touch? *If no:* the charter says I am not allowed to write the Python until the LEAN4 reference exists. Stop and flag.
- ☐ Does `lake build` succeed on the current commit (the LEAN4 reference type-checks)?
- ☐ Does the Python differential test pass against the LEAN4 binary on the current commit (`pytest tests/test_lean4_diff.py` is green)?
- ☐ Have I written down — in plain English — the type, role, and pipeline position of every variable I am about to reason about? (Per the PRIMARY RULE in `feedback_no_fabricated_bugs.md`.)
- ☐ If I am about to claim a bug: do I have a reproducer (input → both implementations → disagreement beyond the tolerance threshold)? *If no:* it is not a bug, it is an open question; record it as such and move on.

If any box is unchecked, the agent does not proceed. The session is paused; the user is notified; the missing artefact is produced before any code is changed.

9 Closing

The point of this document is accountability. The named incident in §2.2 was caused by an agent (me) pattern-matching on `theta` across two architectural layers it does not unify. The cost was burned trust and burned time. The charter prescribed here makes that class of error structurally impossible: the LEAN4 type-checker is unforgiving, and disagreements between LEAN4 and Python are by construction Python bugs.

LEAN4 is never wrong about types. Python is. Agents pattern-match; LEAN4 does not. This charter is the bridge that makes the agent's limitations harmless.